BIRLA INSTITUTE OF TECHNOLOGY, MESRA, RANCHI
(END SEMESTER EXAMINATION)

CLASS: BTech
BRANCH: CS

SEMESTER : VI/Add
SESSION : SP/2024

SUBJECT: CS333/CS305 COMPILER DESIGN

TIME: 3 Hours

FULL MARKS: 50

INSTRUCTIONS:
1. The question paper contains 5 questions each of 10 marks and total 50 marks.
2. Attempt all questions.
3. Before attempting the question paper, be sure that you have got the correct question paper.
4. Answers without justification will get reduced credit.
5. All programs used in a question are given to be error free, unless stated otherwise. Missing data, if any, may be assumed suitably.
6. SDTS for some constructs and caller callee conventions are given at the end. You may adopt them after ensuring their correctness at your own risk.

		Marks	CO	BL
Q.1(a)	Consider the following code fragment in C.	[5]	1,2	1,4

C Declarations	Answer the Questions

const int i = 5;
const int* p=&i;
int*const q=&i;
const int *const r;
	(i) Identify all the distinct lexemes in the code given in Column 1 and group them into tokens. Write the token along with the set of lexemes it represents. **(2 marks)**
	(ii) Write regular expressions in lex notation for specifying all the tokens reported in part (i) above. **(2 marks)**
	(iii) Identify errors in the code given above, lexical, syntax or semantic errors, if any, and report them with reasons. **(1 mark)**

Q.1(b)	Lex specification for floating point numbers in C, written by some student, is given below.	[5]	1,2	1,4

Regular Expression	Answer the Questions
digit [0-9] float {digit}* \ . {digit}*(e{digit}*)?	(i) Find real numbers, if any, that are missed by the given specification (ii) Non-real numbers that match the specification. (iii) Correct the specification so that all floating point values are matched.

Q.2(a)	Consider the properties of a few **binary** operators in some hypothetical language L, as depicted in the following table. Note that these properties are **NOT** necessarily the same as C. The terminal set T has 14 symbols, 11 operators listed in the table, ID and the parenthesis, '(' and ')'.	[5]	1,3	5

Precedence	Associativity	Operator	Function
8	Left	+, -	add-op
7	Right	*, %	mult-op
6	Right	==, !=	equal-op
5	Right	<, >	rel-op
4	Left	&&, ||	logical-op
3	Right	=	assign-op

Construct a parse tree that honours all the given properties of operators.
a = b * c != c != d + e < f && g ;
Show the result of evaluation at each node of the parse tree, given that b =5; c=6; d =1; e =2; and f = g = 3. The Boolean literals true and false have their values as in C.

Q.2(b)	Write an unambiguous CFG to generate sentences in the language L of Question 2.a. Your grammar must comply with all the properties of operators (including parentheses) listed in the table. Use nonterminal from the symbols (A, B, C, D, E, F, G, H).	[5]	1,3	5

PTO

Q.3(a) Examine the following CFG G = (T, N, P, S) where T has 3 tokens {**id**, =, *} with 3 nonterminals, 6 production rules and start symbol S. [5] 2,4 4,5

Grammar G	
1, 2): S → L = R | R 3, 4): L → * R | R 5, 6): R → * L | **id**	(i) Construct LL(1) Parsing table for G. (ii) Is G a LL(1) grammar ? Justify your answer.

Q.3(b) Consider the following left recursive grammar given below. [5] 2,4 4,5

E → E + T | T
T → id | L |
L → L, E | id |E

It is required to write a recursive descent parser for this grammar that works properly without any error. Write the recursive procedure for nonterminal L only.

Q.4(a) Partial and complete evaluation are two different schemes for generating intermediate code for boolean expressions. A boolean expression of the form, **e1 op e2 op e3 op e4** where **op** is one of the binary operators { <, <=, >, >=, ==, !=, &&, || } is given to you. The symbols e1, e2, e3, e4 are expressions, may involve arithmetic operators '+' and '*', but do not contain the operators **op** listed above. The SDTS provided at the end may be examined for errors and corrected if necessary, before use. [5] 4 3,6

(a) Write your version of SDTS for full and ~~complete~~ partial evaluation of boolean expressions. In case you do not wish to make any change to the SDTS provided, mention the same with reasons.(2)

(b) What should be the 3 operators and in which order in the expression **e1 op e2 op e3 op e4** so that the difference in the codes generated is maximum in terms of code size and/or number of incomplete jump targets ?(3)

Q.4(b) Using detail of Question 4.a, write the intermediate codes when full and partial evaluation are used respectively for the following expression and compare the codes in terms of size and number of incomplete jump targets. Note that prec(>=) > prec(&&) > prec (||). [5] 4 3,6

e1 && e2 >= e3 | | e4

Q.5(a) (i)A new construct **repeat S until B**, is to be added to the SDTS for control structures. The semantics of this loop is that the body S is executed at least once. Subsequent execution of the body S depends on the boolean expression B, in case B evaluates to True, the loop is exited. [5] 5 4,6

S →**repeat** S_1 **until B**

Write a SDTS for repeat-until. Do not bother to change the given SDTS for the other constructs unless it is inevitable. If you create new attributes you must justify them. You have to write all the corrections, if any, that you make to the given SDTS with reasons for the change made. **(2 marks)**

(ii) The X 86 64 bit assembly code generated by gcc for the main() function in column 1 is given in column2.

Source program	Assembly code

int c = 4;
static float val = 5.0;
int f(float x) { return x*x - 2.0;}
int main()
{ int a, b;
a = b + c;
printf ("&c = %p &a = %p &b = %p &val = %p\n
&main %p &f %p \n", &c, &a, &b, &val, main, f);
return 0;
}
	.LC1:
	.string "&c = %p &a = %p &b = %p &val = %p\n &main %p &f %p \n"
	main:
	endbr64
	pushq %rbp #
	movq %rsp, %rbp #,
	subq $16, %rsp #,
	movq %fs:40, %rax #
	movq %rax, -8(%rbp) #
	tmp91, D.2330
	xorl %eax, %eax #
	tmp91

```
movl    -12(%rbp), %edx
        # b, b.0_1
movl    c(%rip), %eax    # c,
c.1_2
addl    %edx, %eax       #
b.0_1, _3
movl    %eax, -16(%rbp)
        # _3, a
leaq    -12(%rbp), %rdx#,3
tmp87
leaq    -16(%rbp), %rax#,
tmp88
subq    $8, %rsp         #,
leaq    f(%rip), %rcx -4 #,
tmp89
pushq   %rcx     # tmp89
leaq    main(%rip), %r9#,b
leaq    val(%rip), %r8   #,5
movq    %rdx, %rcx       #
tmp87,
movq    %rax, %rdx       #
tmp88,
leaq    c(%rip), %rsi    #, 2
leaq    .LC1(%rip), %rdi #
movl    $0, %eax         #,
call    printf@PLT       #
addq    $16, %rsp        #,
movl    $0, %eax         #, _7
movq    -8(%rbp), %rsi   #
D.2330, tmp92
xorq    %fs:40, %rsi     #
leave
ret
```

You have to identify in the assembly code the statements that are used by mail to pass the actual arguments to print() function. **(3 marks)**

Q.5(b) ✓

(I) Consider the main program given in 5(a) part (ii). The output generated by this program is manipulated intentionally and given below. [5] 5 4,6

&c = 0x556224db3010 &a = 0x556224db3014 &b = 0x7ffeba810f44
&val = 0x7ffeba810f40 &main = 0x556224db0195 &f = 0x7ffeba810f48

You have to identify which addresses are definitely wrong by virtue of data layout under runtime environment with justifications. Note that you do not have to correct the addresses but indicate which ones are wrong and approximate locations where should have been. **(1 mark)**

(II) Consider the C program listed in Column1, which is given to be correct and the output produced is as expected. The following optimizations, Constant folding, Constant propagation, Unreachable code removal, loop invariant code movement, removing dead code and any other that you may wish to define, are candidates for applying on the given source. **(4 marks)**

(i) Apply as many optimization as you can and show the result in the following format: Name of the optimization; code fragment to which applied; Why it is safe to use it; changed source code before and after the optimization for the affected code only. The sequence in which the optimizations are to be applied is to be decided by you.

(ii) Show the final optimized source code.

C Source Program

```
int main()
{ int a = 2, b = 3, c = 40, d = 5, i, j;
 int x[11];
 for (j = 0; j < 11; j++) scanf("%d ",&x[j]);
 for ( i = 0; i < 11; i+=2 )
 { a++;
  while (i%2 ==1) {a = a * b; x[i] = x[i+1] + d; c = 10*b; }
  a = a * b; d = c * c; x[(i+d)%200] = x[i]+x[i-1];
 };
 if (a < 2) for (j = 1; j < 11; j++) x[j]=x[j]+5;
 else { x[5]+= d; printf(" a = %d x[5] = %d d = %d\n", a, x[5], d); return 5;
      while (a + b > 10) {a++; b++;}
      };
 return 0;
}
```

Appendix A:

In the SDTS given below, some errors may have been intentionally inserted. Detect, correct and report the errors with reasons in your answers.

Translation of Basic Assignment Statement

$S \rightarrow id := E$	{ p := lookup (id.name); if p ≠ nil then emit (p ':=' E.place) else error /* undeclared id */ }
$E \rightarrow E_1 + E_2$	{E.place := newtemp(); emit (E.place ':=' E_1.place '+' E_2.place)}
$E \rightarrow E_1 * E_2$	{E.place := newtemp(); emit (E.place ':=' E_1.place '*' E_2.place)}
$E \rightarrow - E_1$	{E.place := newtemp(); emit (E.place ':=' 'uminus' E_1.place) }
$E \rightarrow (E_1)$	{ E.place := E_1.place) }
$E \rightarrow$ id	{ p := lookup (id.name); if p ≠ nil then E.place := p else error /* undeclared id */

SDTS for Full Evaluation of Boolean expressions

$B \rightarrow B_1$ **or** B_2	{ B.place := newtemp(); emit(B.place ':=' B_1.place 'or' B_2.place) }
$B \rightarrow B_1$ **and** B_2	{ B.place := newtemp(); emit(B.place ':=' B_1.place 'and' B_2.place) }
$B \rightarrow$ **not** B_1	{ B.place := newtemp(); emit(B.place ':=' 'not' B_1.place) }
$B \rightarrow (B_1)$	{ B.place := B_1.place) }
$B \rightarrow$ **true**	{ B.place := newtemp(); emit (B.place ':=' '1') }
$B \rightarrow$ **false**	{ B.place := newtemp(); emit (B.place ':=' '0') }
$B \rightarrow E$	{ B.place := E.place}
$B \rightarrow E_1$ **relop** E_2	{ B.place := newtemp(); emit ('if E_1.place 'relop.op' E_2.place 'goto' nextstat+4); emit (B.place ':=' '0'); emit ('goto' nextstat + 2); emit (B.place ':=' '1')}

Scheme2 : Partial evaluation of boolean expressions :

$B \rightarrow B_1$ **or** $M\ B_2$ { backpatch (B_1.falselist, M.stat); B.truelist := merge (B_1.truelist, B_2.truelist); B.falselist := B_2.falselist}

$B \rightarrow B_1$ **and** $M\ B_2$ { backpatch (B_1.truelist, M.stat); B.falselist := merge (B_1.falselist, B_2.falselist); B.truelist := B_2.truelist}

$B \rightarrow$ **not** B_1 { B.truelist := B_1.falselist ; B.falselist := B_1.truelist }

$B \rightarrow (B_1)$ { B.truelist := B_1.truelist ; B.falselist := B_1.falselist }

$B \rightarrow$ **true** { B.truelist := makelist (nextstat); emit ('goto ___');

$B \rightarrow$ **false** { B.falselist := makelist (nextstat); emit ('goto ___');

$B \rightarrow E_1$ **relop** E_2 { B.truelist := makelist (nextstat); B.falselist := makelist (nextstat + 1); emit ('if' E_1.place 'relop.op' E_2.place 'goto ___'); emit ('goto ___') }

$B \rightarrow E$ { B.place := E.place; B.truelist := makelist (nextstat); B.falselist := makelist (nextstat + 2);

emit ('if' B.place 'goto ___'); emit ('goto ___') }

M → ε {M.stat := nextstat}

E → E_1 + E_2 | other rules for expressions { semantic actions as discussed earlier }

4. SDTS for control flow constructs

S → if B then M S_1 {backpatch (B.truelist, M.stat) ; S.nextlist := merge(B.falselist, S_1.nextlist) }

S → if B then M_1 S_1 N else M_2 S {backpatch (B. truelist, M_1.stat) ; backpatch (B.falselist, M_2.stat); S.next := merge(S_1.nextlist, S_2.nextlist)}

S → while M_1 B do M_2 S_1 {backpatch (B.truelist, M_2.stat); backpatch (S_1.nextlist, M_1.stat); S.nextlist := B.falselist ;}

S → begin Sl end {S.nextlist := Sl.nextlist}

Sl → Sl_1 ; M S {backpatch (Sl_1.nextlist, M.stat); Sl.nextlist := S.nextlist}

Sl → S {Sl.nextlist := S.nextlist}

S → L := E { semantic actions as earlier for assignment ; S.nextlist := nil}

M → ε {M.stat := nextstat}

N → ε {N.nextlist := makelist (nextstat); emit ('goto ___ ')}

Generic Caller -Callee Conventions : details may vary from architecture to architecture

Procedure Call		Return from a procedure	
Tasks done by Caller	Tasks done by Callee	Tasks done by Caller	Tasks done by Callee
1. Makes space on the stack for a return value. 2. Puts the actual parameters on the stack. 3. Sets the static link. 4. Jumps to the called procedure. Return address is saved on the stack.	1. Sets the dynamic link. 2. Sets the base of the new activation record. 3. Saves registers on the stack. 4. Makes space for local variables on the stack.	1. Restores stackpointer to the location containing the returned value.	1. Restores registers. 2. Sets the base pointer to the activation record of the calling procedure. 3. Returns to the caller.

Selected Registers of X86 and their use

%rsp Stack Pointer	%rax Return value	%rdi 1st argument	%rsi 2nd argument
%rdx 3rd argument	%rcx 4th argument	%r8 5th argument	%r9 6th argument
%rip instruction pointer	%rbp base pointer		

:::::25/04/2024 M::::